TechStyle Ecommerce

Data Modeling

Modelagem relacional avançada para e-commerce · PostgreSQL

1. Problema de Negócio

1.1 Contexto

A TechStyle é um e-commerce com operação nacional, múltiplos centros de distribuição e uma base crescente de clientes. A empresa enfrenta um desafio clássico de escala: à medida que o volume de pedidos aumenta, os dados críticos de vendas, logística e comportamento do cliente ficam espalhados em sistemas distintos, tornando impossível uma visão analítica integrada.

A ausência de um modelo de dados coeso gera impactos diretos no negócio:

- Sem snapshots de preço por item, relatórios de margem histórica são imprecisos ou inviáveis
- O histórico de segmentação de clientes se perde, impedindo análises de ciclo de vida (CLV) e churn
- Devoluções não são rastreadas no nível do item, tornando difícil identificar produtos problemáticos
- A atribuição de campanhas a pedidos é inconsistente, comprometendo o cálculo de ROI de marketing

1.2 Objetivos do Modelo

O modelo foi projetado para viabilizar as seguintes análises de negócio de forma direta, sem transformações complexas:

Área	Perguntas que o modelo responde
Vendas & Receita	Qual a receita líquida por período, categoria e canal? Qual a margem por produto?
Clientes	Qual o LTV por segmento? Quais clientes migraram de Novo para VIP nos últimos 6 meses?
Logística	Qual o tempo médio de entrega por transportadora? Quantos pedidos estão atrasados?

Área	Perguntas que o modelo responde
Marketing	Qual o ROI por campanha? Qual cupom gerou mais conversões?
Devoluções	Quais produtos têm maior taxa de devolução? Qual o motivo predominante?
Estoque	Quais produtos estão abaixo do nível de reposição em cada CD?

1.3 Escopo

Banco de dados: PostgreSQL · 17 tabelas · ~65 colunas rastreadas · Cobertura: clientes, produtos, pedidos, pagamentos, logística, devoluções, avaliações e campanhas

2. Diagrama de Fluxo de Dados

O diagrama abaixo representa o ciclo de vida completo de um pedido no sistema TechStyle — desde o cadastro do cliente até a eventual devolução e avaliação. O fluxo mostra quais tabelas são escritas em cada etapa e como os dados se propagam entre os domínios do modelo.

2.1 Ciclo de Vida do Pedido

8	**Devolução (opcional)** `returns · payments (refund)`

Caminhos alternativos: Pagamento reprovado → retentativa em payments (status: failed → nova linha). Pedido cancelado antes do envio → orders.status = cancelled, sem registro em shipments. Devolução parcial → múltiplos registros em returns, um por order_item_id devolvido.

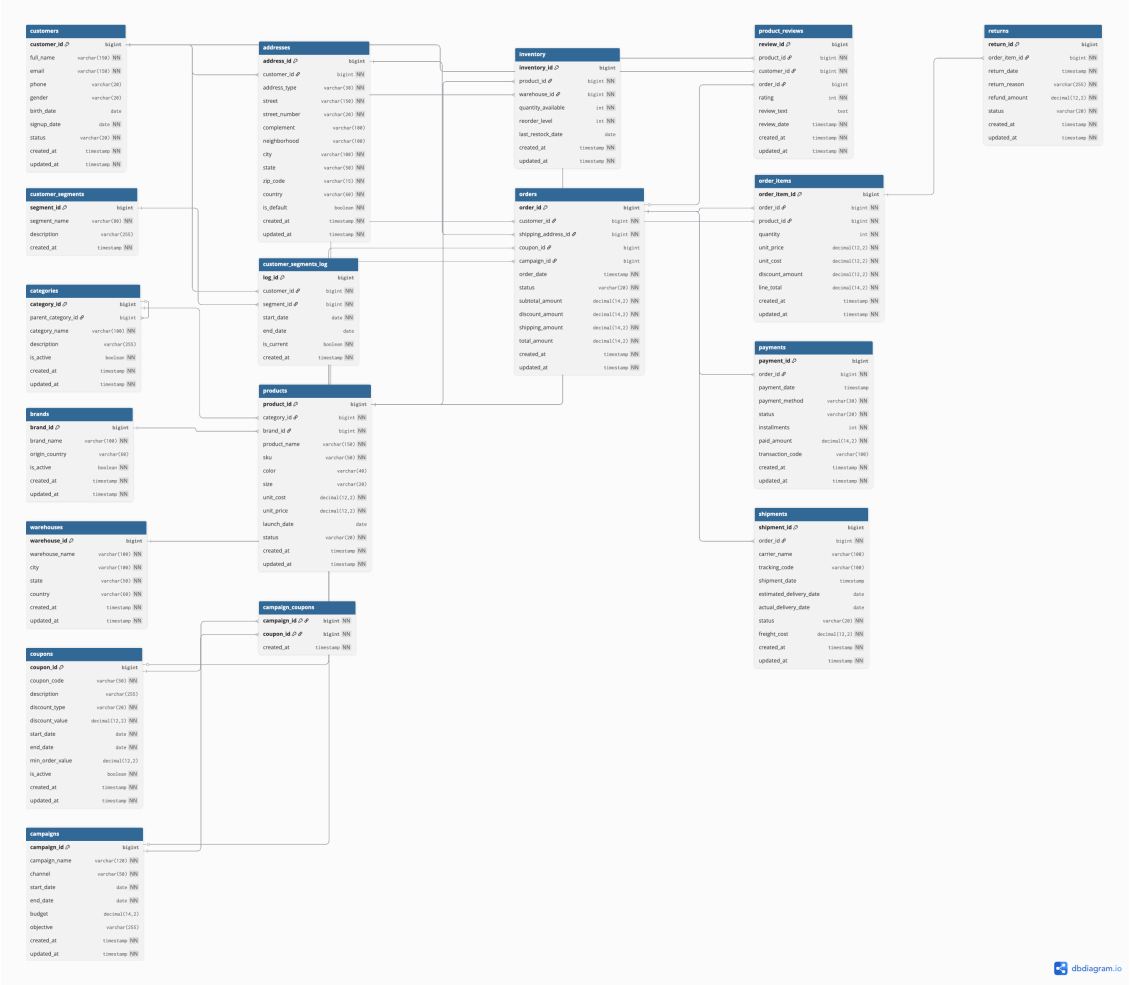
2.2 Fluxo de Dados de Marketing

O diagrama abaixo mostra como uma campanha de marketing se conecta ao modelo transacional e como o impacto financeiro é rastreado de ponta a ponta:

A	**Campanha criada** `campaigns — channel, budget, período`
B	**Cupons associados** `campaign_coupons → coupons`
C	**Cliente converte** `orders.campaign_id + orders.coupon_id preenchidos`
D	**Receita atribuída** `SUM(orders.total_amount) WHERE campaign_id = X`
E	**ROI calculado** `receita_gerada / campaigns.budget`

3. Documentação do ERD

Documentação de referência de todas as 17 tabelas do modelo. Para cada tabela são descritos: propósito, colunas relevantes, enumerações e relacionamentos.



https://dbdiagram.io/d/TechStyle-69d6e7e88089629684504b9c

3.1 Domínio — Clientes & Segmentação

customers

Cadastro principal dos clientes. Ponto de entrada para toda análise de comportamento, LTV e segmentação.

Coluna	Descrição
customer_id	PK — identificador único do cliente (bigint, auto-increment)
full_name	Nome completo
email	E-mail único — usado como login e chave de deduplicação

Coluna	Descrição
phone	Telefone opcional
gender / birth_date	Dados demográficos para análises de perfil
signup_date	Data de cadastro — base para cálculo de idade do cliente
status	Enum: active | inactive | blocked

Relacionamentos: 1:N com orders, addresses, customer_segments_log, product_reviews

customer_segments

Tabela de domínio dos segmentos disponíveis. Exemplos típicos: Novo, Recorrente, VIP, Inativo.

Coluna	Descrição
segment_id	PK
segment_name	Nome único do segmento — ex.: 'VIP', 'Novo', 'Recorrente'
description	Descrição dos critérios de enquadramento nesse segmento

Relacionamentos: 1:N com customer_segments_log

customer_segments_log

Histórico de transições de segmento por cliente. Cada linha representa um período em que o cliente pertenceu a um determinado segmento. Viabiliza análises de ciclo de vida e detecção de churn.

Coluna	Descrição
log_id	PK
customer_id	FK → customers
segment_id	FK → customer_segments
start_date	Início da vigência no segmento
end_date	Fim da vigência — NULL se for o segmento atual
is_current	Flag booleana: true apenas para o segmento ativo do cliente

Padrão de uso: Para obter o segmento atual: WHERE is_current = true. Para histórico: JOIN por período usando start_date / end_date.

addresses

Endereços associados a um cliente. Um cliente pode ter múltiplos endereços; apenas um pode ser marcado como padrão (is_default = true). A tabela é referenciada por orders para registrar o endereço de entrega no momento do pedido.

Coluna	Descrição
address_id	PK
customer_id	FK → customers
address_type	Enum: billing | shipping | both
street / street_number / complement	Logradouro completo
city / state / zip_code / country	Localização — country padrão: Brasil
is_default	true = endereço principal do cliente

Relacionamentos: N:1 com customers · 1:N com orders (via shipping_address_id)

3.2 Domínio — Catálogo de Produtos

categories

Hierarquia de categorias do catálogo. A coluna parent_category_id aponta para a própria tabela (self-join), permitindo árvores de qualquer profundidade. Queries recursivas com WITH RECURSIVE percorrem a hierarquia.

Coluna	Descrição
category_id	PK
parent_category_id	FK → categories (self-referencing) — NULL = categoria raiz
category_name	Nome único da categoria
is_active	false = categoria desativada, não exibida no catálogo

Exemplo de hierarquia: Roupas (root) → Camisetas → Camisetas Polo

brands

Cadastro de marcas. Cada produto é obrigatoriamente associado a uma marca, permitindo análises de desempenho por marca.

Coluna	Descrição
brand_id	PK
brand_name	Nome único da marca
origin_country	País de origem — opcional, usado em filtros e relatórios
is_active	false = marca desativada

Relacionamentos: 1:N com products

products

Catálogo de SKUs. Cada linha representa uma variação única de produto (cor + tamanho). Os campos unit_cost e unit_price são os valores vigentes no catálogo — os valores históricos de vendas ficam em order_items.

Coluna	Descrição
product_id	PK
sku	Código único de identificação do produto — chave natural
category_id	FK → categories
brand_id	FK → brands
color / size	Atributos de variação do produto
unit_cost	Custo unitário atual (catálogo)
unit_price	Preço de venda atual (catálogo)
launch_date	Data de lançamento — usado em análises de curva de vida
status	Enum: active | inactive | discontinued

Relacionamentos: 1:N com order_items, inventory, product_reviews

warehouses

Centros de distribuição onde o estoque é armazenado. Permite análises de cobertura geográfica e desempenho logístico por CD.

Coluna	Descrição
warehouse_id	PK
warehouse_name	Nome do centro de distribuição
city / state / country	Localização do CD

Relacionamentos: 1:N com inventory

inventory

Posição de estoque por produto e centro de distribuição. O índice único em (product_id, warehouse_id) garante que cada combinação produto-CD tenha apenas uma linha. O campo reorder_level define o ponto de reposição.

Coluna	Descrição
inventory_id	PK
product_id	FK → products
warehouse_id	FK → warehouses
quantity_available	Quantidade disponível para venda
reorder_level	Quantidade mínima — abaixo disso aciona reposição
last_restock_date	Data do último recebimento de estoque

Índice único: (product_id, warehouse_id) — garante uma linha por produto por CD

3.3 Domínio — Marketing

campaigns

Campanhas de marketing com canal, período e orçamento. Pedidos podem ser atribuídos a uma campanha via orders.campaign_id, viabilizando cálculo de ROI direto.

Coluna	Descrição
campaign_id	PK
campaign_name	Nome da campanha
channel	Canal de mídia — ex.: google_ads, instagram, email
start_date / end_date	Período de veiculação
budget	Investimento total da campanha
objective	Descrição do objetivo — ex.: aquisição, retenção, reativação

Relacionamentos: 1:N com orders · N:N com coupons via campaign_coupons

coupons

Cupons de desconto com tipo (percentual ou fixo), período de validade e valor mínimo de pedido. Podem ser associados a campanhas via campaign_coupons ou aplicados diretamente em pedidos.

Coluna	Descrição
coupon_id	PK
coupon_code	Código único digitado pelo cliente no checkout
discount_type	Enum: percentage | fixed
discount_value	Valor do desconto — percentual (0-100) ou fixo (R$)
start_date / end_date	Período de validade do cupom
min_order_value	Valor mínimo do pedido para aplicação do cupom — nullable
is_active	false = cupom desativado manualmente antes do vencimento

Relacionamentos: 1:N com orders · N:N com campaigns via campaign_coupons

campaign_coupons

Tabela de junção entre campaigns e coupons. PK composta em (campaign_id, coupon_id) — padrão para junction tables sem atributos próprios além de created_at.

Coluna	Descrição
campaign_id	PK (composta) + FK → campaigns
coupon_id	PK (composta) + FK → coupons
created_at	Timestamp de associação

3.4 Domínio — Pedidos & Itens

orders

Cabeçalho do pedido. Concentra os totais financeiros, o endereço de entrega snapshot, o cupom aplicado e a campanha de origem. O status percorre o ciclo de vida do pedido da criação à entrega.

Coluna	Descrição
order_id	PK
customer_id	FK → customers

Coluna	Descrição
shipping_address_id	FK → addresses — snapshot do endereço no momento do pedido
coupon_id	FK → coupons — nullable
campaign_id	FK → campaigns — nullable, atribuição de origem
order_date	Timestamp do momento em que o pedido foi criado
status	Enum: pending | paid | shipped | delivered | cancelled
subtotal_amount	Soma dos line_total dos itens antes de descontos e frete
discount_amount	Total de descontos aplicados (cupom + promoções)
shipping_amount	Valor do frete cobrado
total_amount	subtotal - discount + shipping — valor final pago

Relacionamentos: 1:N com order_items · 1:1 com payments e shipments · 1:N com product_reviews

order_items

Linhas do pedido. Cada linha representa um SKU dentro de um pedido. Os campos unit_price e unit_cost são copiados do catálogo no momento da venda — esse snapshot histórico é o que torna possível calcular margem real para qualquer período passado.

Coluna	Descrição
order_item_id	PK
order_id	FK → orders
product_id	FK → products
quantity	Quantidade do SKU nesse item
unit_price	Preço unitário no momento da venda (snapshot)
unit_cost	Custo unitário no momento da venda (snapshot)
discount_amount	Desconto alocado a este item
line_total	quantity × unit_price − discount_amount

Relacionamentos: N:1 com orders e products · 1:N com returns

3.5 Domínio — Transações

payments

Registro de pagamentos. Cada linha representa uma tentativa ou pagamento efetivado. A ausência de unique em order_id é intencional — suporta retentativas e prepara o modelo para split payments futuros. A lógica de um único pagamento aprovado por pedido deve ser garantida pela aplicação.

Coluna	Descrição
payment_id	PK
order_id	FK → orders — sem unique (permite retentativas)
payment_date	Timestamp da confirmação do pagamento
payment_method	Enum: credit_card | debit_card | pix | boleto
status	Enum: pending | approved | failed | refunded
installments	Número de parcelas — padrão 1 (à vista)
paid_amount	Valor efetivamente pago
transaction_code	Código do gateway de pagamento para rastreabilidade

shipments

Dados de entrega do pedido. Unique em order_id — relação 1:1 com orders. Permite calcular SLA de entrega, percentual de entregas no prazo e desempenho por transportadora.

Coluna	Descrição
shipment_id	PK
order_id	FK → orders — unique (1 remessa por pedido)
carrier_name	Nome da transportadora
tracking_code	Código de rastreamento
shipment_date	Timestamp de despacho
estimated_delivery_date	Prazo prometido ao cliente
actual_delivery_date	Data real de entrega
status	Enum: preparing | shipped | delivered | delayed | returned
freight_cost	Custo do frete para a empresa

returns

Devoluções no nível do item — referencia order_item_id, não order_id. Essa granularidade permite identificar SKUs com alta taxa de devolução e cruzar com avaliações para correlacionar qualidade percebida com devoluções.

Coluna	Descrição
return_id	PK
order_item_id	FK → order_items — granularidade por item, não por pedido
return_date	Timestamp da solicitação de devolução
return_reason	Motivo descritivo — ex.: 'tamanho errado', 'produto danificado'
refund_amount	Valor a ser reembolsado ao cliente
status	Enum: requested | approved | rejected | refunded

3.6 Domínio — Avaliações

product_reviews

Avaliações de produtos feitas por clientes. A FK order_id é nullable — permite avaliações não verificadas (sem compra associada). Quando preenchida, garante que a avaliação é de um comprador real, o que pode ser usado como filtro de confiabilidade.

Coluna	Descrição
review_id	PK
product_id	FK → products
customer_id	FK → customers
order_id	FK → orders — nullable (avaliação não verificada se NULL)
rating	Nota de 1 a 5
review_text	Texto livre da avaliação — nullable
review_date	Timestamp da avaliação

> **Padrão de uso:** JOIN com returns para correlacionar notas baixas (rating ≤ 2) com devoluções do mesmo produto.

3.7 Mapa de Relacionamentos

Referência consolidada de todas as chaves estrangeiras do modelo:

Chave estrangeira	Referência
customer_segments_log.customer_id	customers.customer_id
customer_segments_log.segment_id	customer_segments.segment_id

Chave estrangeira	Referência
addresses.customer_id	customers.customer_id
categories.parent_category_id	categories.category_id (self)
products.category_id	categories.category_id
products.brand_id	brands.brand_id
inventory.product_id	products.product_id
inventory.warehouse_id	warehouses.warehouse_id
campaign_coupons.campaign_id	campaigns.campaign_id
campaign_coupons.coupon_id	coupons.coupon_id
orders.customer_id	customers.customer_id
orders.shipping_address_id	addresses.address_id
orders.coupon_id	coupons.coupon_id
orders.campaign_id	campaigns.campaign_id
order_items.order_id	orders.order_id
order_items.product_id	products.product_id
payments.order_id	orders.order_id
shipments.order_id	orders.order_id
returns.order_item_id	order_items.order_item_id
product_reviews.product_id	products.product_id
product_reviews.customer_id	customers.customer_id
product_reviews.order_id	orders.order_id

4. Explicação Técnica do Modelo

4.1 Visão Geral da Arquitetura

O modelo segue o paradigma relacional normalizado (3FN), organizado em cinco domínios funcionais com fronteiras bem definidas. Cada domínio concentra responsabilidades específicas, o que facilita tanto a manutenção quanto a construção de queries analíticas.

Domínio	Tabelas
Clientes & Segmentação	customers, customer_segments, customer_segments_log, addresses
Catálogo de Produtos	products, categories, brands, warehouses, inventory

Domínio	Tabelas
Pedidos & Itens	orders, order_items
Transações	payments, shipments, returns
Marketing	campaigns, coupons, campaign_coupons

4.2 Decisões de Design

Snapshot de preço e custo em order_items

As colunas unit_price e unit_cost são copiadas do catálogo no momento da venda. Isso garante que relatórios históricos de margem reflitam o valor real praticado, independentemente de alterações futuras no preço do produto — uma das armadilhas mais comuns em modelos de e-commerce.

Histórico de segmentação (customer_segments_log)

Em vez de manter apenas o segmento atual no cadastro do cliente, o modelo persiste cada transição com start_date, end_date e is_current. Isso viabiliza análises temporais como: 'quantos clientes migraram do segmento Novo para VIP nos últimos 90 dias?' — algo impossível com apenas um campo de segmento na tabela de clientes.

Devolução no nível do item (returns → order_items)

A tabela returns referencia order_item_id, não order_id. Essa granularidade permite identificar exatamente qual SKU foi devolvido, calcular taxa de devolução por produto e cruzar com avaliações para correlacionar baixas notas com devoluções.

Junction table com PK composta (campaign_coupons)

A tabela de associação entre campanhas e cupons usa PK composta (campaign_id, coupon_id), eliminando a necessidade de uma coluna surrogate. Isso é o padrão para tabelas de junção puras sem atributos próprios além de created_at.

payments sem unique em order_id

A restrição unique foi intencionalmente removida de payments.order_id para suportar retentativas de pagamento e preparar o modelo para split payments. A lógica de '1 pedido, 1 pagamento aprovado' deve ser garantida pela aplicação ou por uma constraint parcial (WHERE status = 'approved').

Categorias hierárquicas (self-join)

A coluna parent_category_id aponta para a própria tabela categories, permitindo hierarquias como Roupas > Camisetas > Camisetas Polo sem limite de profundidade. Queries recursivas com CTEs (WITH RECURSIVE) percorrem essa hierarquia de forma eficiente.

4.3 Padrões de Auditoria

Todas as tabelas possuem created_at e updated_at (timestamp), garantindo rastreabilidade completa de quando cada registro foi criado e modificado. Tabelas de histórico (customer_segments_log) adicionam colunas de vigência (start_date, end_date) para suportar análises temporais sem precisar de tabelas de auditoria separadas.

5. Trade-offs e Decisões Rejeitadas

Todo modelo é uma série de escolhas — e as decisões rejeitadas revelam o raciocínio por trás do que ficou. Esta seção documenta as principais alternativas avaliadas, por que foram consideradas e por que foram descartadas.

5.1 Histórico de segmento: coluna na tabela de clientes vs. tabela dedicada

Opção rejeitada	Opção adotada
Adicionar segment_id diretamente em customers	Tabela customer_segments_log com start_date, end_date e is_current

A coluna simples em customers diz apenas onde o cliente está hoje. A tabela de log preserva toda a trajetória: quando o cliente entrou em cada segmento, quanto tempo ficou e quando saiu. Isso habilita perguntas de negócio críticas como taxa de conversão Novo → VIP, tempo médio para upgrade e identificação de clientes em risco de regredir de segmento — análises impossíveis sem o histórico.

O custo dessa decisão é uma join adicional em queries de segmento atual, mitigado pelo índice em (customer_id, is_current).

5.2 Devolução: nível do pedido vs. nível do item

Opção rejeitada	Opção adotada
returns.order_id — devolução por pedido inteiro	returns.order_item_id — devolução por item individual

Clientes frequentemente devolvem apenas parte de um pedido. Com a FK em order_id, cada devolução parcial exigiria uma tabela auxiliar de itens devolvidos de qualquer forma — tornando a abordagem mais complexa sem ganhar nada. A FK direta em order_item_id é mais granular, mais simples e viabiliza diretamente a taxa de devolução por SKU, cruzamento com avaliações e cálculo de reembolso preciso por item.

5.3 Tabela de auditoria genérica vs. tabelas de log especializadas

Opção rejeitada	Opção adotada
audit_log genérico com entity_type, entity_id, old_value, new_value (EAV)	customer_segments_log especializado com colunas tipadas e período de vigência explícito

O padrão EAV (Entity-Attribute-Value) em tabelas de auditoria genéricas é flexível, mas transforma toda query analítica em um pivot complexo sobre strings. Para o caso de uso central — análise de ciclo de vida de clientes — uma tabela tipada com start_date e end_date permite JOINs diretos, filtros por período e agregações sem nenhuma transformação intermediária. A especialização tem custo de manutenção de schema, mas o ganho em legibilidade e performance das queries justifica amplamente.

5.4 Pagamentos: unique constraint vs. restrição por aplicação

Opção rejeitada	Opção adotada
payments.order_id UNIQUE — força 1 pagamento por pedido no banco	Sem unique — constraint parcial sugerida: UNIQUE (order_id) WHERE status = 'approved'

Um UNIQUE simples impede retentativas de pagamento (falha de rede → tentativa 2 → bloqueado) e inviabiliza split payments futuros. A constraint parcial (unique apenas para status = 'approved') preserva a integridade sem bloquear o fluxo operacional — um padrão PostgreSQL que combina segurança de banco de dados com flexibilidade de negócio.

5.5 Categoria de produto: coluna de texto vs. hierarquia relacional

Opção rejeitada	Opção adotada
products.category VARCHAR com valor composto — ex.: 'Roupas/Camisetas/Polo'	Tabela categories com self-join via parent_category_id — hierarquia relacional

Colunas de texto com hierarquia embutida (path strings) exigem LIKE ou funções de string para qualquer query analítica por nível — frágeis, lentas e difíceis de renomear. A hierarquia

relacional com parent_category_id permite consultas recursivas limpas com WITH RECURSIVE, renomeação sem efeito cascata e fácil adição de novos níveis sem alterar o schema.

6. Principais Queries

6.1 Receita e Margem por Categoria

Calcula receita líquida (descontando devoluções), custo total e margem bruta por categoria no mês corrente:

```
SELECT
    c.category_name,
    SUM(oi.line_total)                              AS receita_bruta,
    COALESCE(SUM(r.refund_amount), 0)              AS total_devolucoes,
    SUM(oi.line_total) - COALESCE(SUM(r.refund_amount), 0)
                                                   AS receita_liquida,
    SUM(oi.quantity * oi.unit_cost)                AS custo_total,
    ROUND(
      (SUM(oi.line_total) - SUM(oi.quantity * oi.unit_cost))
      / NULLIF(SUM(oi.line_total), 0) * 100, 2
    )                                              AS margem_pct
FROM order_items oi
JOIN orders      o  ON o.order_id      = oi.order_id
JOIN products    p  ON p.product_id    = oi.product_id
JOIN categories  c  ON c.category_id   = p.category_id
LEFT JOIN returns r ON r.order_item_id = oi.order_item_id
WHERE o.status NOT IN ('cancelled')
  AND DATE_TRUNC('month', o.order_date) = DATE_TRUNC('month', NOW())
GROUP BY c.category_name
ORDER BY receita_liquida DESC;
```

6.2 Ranking de Clientes por LTV

Lifetime value por cliente: total gasto, número de pedidos, ticket médio e dias desde o último pedido (recência):

```
SELECT
    cu.customer_id,
    cu.full_name,
    COUNT(DISTINCT o.order_id)          AS total_pedidos,
    SUM(o.total_amount)                 AS ltv,
    ROUND(AVG(o.total_amount), 2)       AS ticket_medio,
    MAX(o.order_date::date)             AS ultimo_pedido,
    NOW()::date - MAX(o.order_date::date)
```

```
                                        AS dias_inativo
FROM customers cu
JOIN orders o ON o.customer_id = cu.customer_id
WHERE o.status = 'delivered'
GROUP BY cu.customer_id, cu.full_name
ORDER BY ltv DESC
LIMIT 100;
```

6.3 ROI de Campanhas

Compara o investimento de cada campanha com a receita gerada pelos pedidos atribuídos a ela:

```
SELECT
    ca.campaign_name,
    ca.channel,
    ca.budget,
    COUNT(DISTINCT o.order_id)              AS pedidos_gerados,
    SUM(o.total_amount)                     AS receita_gerada,
    ROUND(SUM(o.total_amount) / NULLIF(ca.budget, 0), 2)
                                            AS roi
FROM campaigns ca
LEFT JOIN orders o ON o.campaign_id = ca.campaign_id
    AND o.status NOT IN ('cancelled')
GROUP BY ca.campaign_id, ca.campaign_name, ca.channel, ca.budget
ORDER BY roi DESC NULLS LAST;
```

6.4 Produtos com Alta Taxa de Devolução

Identifica os produtos com maior percentual de itens devolvidos em relação ao total vendido:

```
SELECT
    p.sku,
    p.product_name,
    b.brand_name,
    COUNT(oi.order_item_id)                       AS total_vendas,
    COUNT(r.return_id)                            AS total_devolucoes,
    ROUND(
      COUNT(r.return_id)::numeric
      / NULLIF(COUNT(oi.order_item_id), 0) * 100, 1
    )                                             AS taxa_devolucao_pct
FROM order_items oi
JOIN products   p  ON p.product_id    = oi.product_id
JOIN brands     b  ON b.brand_id      = p.brand_id
LEFT JOIN returns r ON r.order_item_id = oi.order_item_id
```

```
GROUP BY p.sku, p.product_name, b.brand_name
HAVING COUNT(oi.order_item_id) >= 10
ORDER BY taxa_devolucao_pct DESC
LIMIT 20;
```

6.5 Desempenho Logístico por Transportadora

Calcula o tempo médio de entrega e o percentual de entregas no prazo por transportadora:

```
SELECT
    s.carrier_name,
    COUNT(*)                                        AS total_entregas,
    ROUND(AVG(
      s.actual_delivery_date - s.shipment_date::date
    ), 1)                                           AS dias_medio_entrega,
    ROUND(
      SUM(CASE WHEN s.actual_delivery_date
                      <= s.estimated_delivery_date
               THEN 1 ELSE 0 END)::numeric
      / COUNT(*) * 100, 1
    )                                               AS pct_no_prazo
FROM shipments s
WHERE s.status = 'delivered'
  AND s.actual_delivery_date IS NOT NULL
GROUP BY s.carrier_name
ORDER BY pct_no_prazo DESC;
```

6.6 Estoque Abaixo do Nível de Reposição

Lista os produtos que precisam de reabastecimento em cada centro de distribuição:

```
SELECT
    w.warehouse_name,
    w.city,
    p.sku,
    p.product_name,
    i.quantity_available,
    i.reorder_level,
    i.reorder_level - i.quantity_available      AS deficit
FROM inventory i
JOIN products   p ON p.product_id   = i.product_id
JOIN warehouses w ON w.warehouse_id = i.warehouse_id
WHERE i.quantity_available < i.reorder_level
  AND p.status = 'active'
ORDER BY deficit DESC;
```

6.7 Migração de Segmento de Clientes

Identifica clientes que migraram de um segmento para outro nos últimos 90 dias, útil para análises de ciclo de vida:

```
SELECT
    cu.full_name,
    s_old.segment_name    AS segmento_anterior,
    s_new.segment_name    AS segmento_atual,
    log_new.start_date    AS data_migracao
FROM customer_segments_log log_new
JOIN customer_segments_log log_old
    ON  log_old.customer_id = log_new.customer_id
    AND log_old.end_date    = log_new.start_date
JOIN customers         cu    ON cu.customer_id  = log_new.customer_id
JOIN customer_segments s_new ON s_new.segment_id = log_new.segment_id
JOIN customer_segments s_old ON s_old.segment_id = log_old.segment_id
WHERE log_new.start_date >= NOW() - INTERVAL '90 days'
ORDER BY log_new.start_date DESC;
```

6.8 Análise RFM Simplificada

Recência, Frequência e Valor Monetário como base para segmentação comportamental:

```
WITH rfm AS (
  SELECT
    customer_id,
    NOW()::date - MAX(order_date::date)  AS recencia_dias,
    COUNT(*)                             AS frequencia,
    SUM(total_amount)                    AS monetario
  FROM orders
  WHERE status = 'delivered'
  GROUP BY customer_id
)
SELECT
    customer_id,
    recencia_dias,
    frequencia,
    ROUND(monetario, 2)    AS monetario,
    NTILE(5) OVER (ORDER BY recencia_dias ASC)  AS r_score,
    NTILE(5) OVER (ORDER BY frequencia DESC)    AS f_score,
    NTILE(5) OVER (ORDER BY monetario DESC)     AS m_score
FROM rfm
ORDER BY m_score DESC, f_score DESC;
```

7. Estimativa de Volume e Considerações de Performance

Um modelo analítico precisa ser pensado para o volume que vai enfrentar em produção. Esta seção estima o crescimento das tabelas centrais e documenta os índices necessários para que as queries da Seção 3 mantenham performance aceitável em escala.

7.1 Estimativa de Volume (cenário: e-commerce médio nacional)

Tabela	Estimativa de crescimento
customers	~5.000 novos cadastros/mês → 60k/ano → 300k em 5 anos
orders	~50.000 pedidos/mês → 600k/ano → 3M em 5 anos
order_items	~2,5 itens/pedido em média → 125k linhas/mês → 7,5M em 5 anos
payments	~1,1 linha/pedido (retentativas) → 55k/mês → 3,3M em 5 anos
shipments	1:1 com orders → 50k/mês → 3M em 5 anos
inventory	Estável — cresce apenas com novos produtos e CDs. Estimativa: ~50k linhas
product_reviews	~8% dos pedidos geram review → 4k/mês → 240k em 5 anos
customer_segments_log	~2 trocas/cliente/ano → 50k linhas/ano → 250k em 5 anos

> **Premissa:** As tabelas orders e order_items são o núcleo analítico do modelo e concentram ~80% do volume total. Toda estratégia de indexação e particionamento deve priorizar essas duas tabelas.

7.2 Índices Recomendados

Os índices abaixo foram definidos com base nas queries da Seção 3 e nos padrões de acesso mais frequentes em sistemas de e-commerce. O objetivo é cobrir os filtros e JOINs mais custosos sem indexação excessiva, que degrada a performance de escrita.

orders — tabela de maior acesso analítico

```
-- Filtro por período (queries de receita, RFM, relatórios mensais)
CREATE INDEX idx_orders_date
    ON orders (order_date DESC);

-- Filtro por cliente (LTV, histórico de pedidos)
CREATE INDEX idx_orders_customer
```

```
    ON orders (customer_id, order_date DESC);

-- Filtro por status (pedidos ativos, cancelamentos)
CREATE INDEX idx_orders_status
    ON orders (status) WHERE status NOT IN ('delivered', 'cancelled');

-- Atribuição de campanha (queries de ROI)
CREATE INDEX idx_orders_campaign
    ON orders (campaign_id) WHERE campaign_id IS NOT NULL;
```

order_items — join central em toda query de produto

```
-- JOIN com orders (acesso mais frequente)
CREATE INDEX idx_order_items_order
    ON order_items (order_id);

-- Análise de vendas por produto (margem, taxa de devolução)
CREATE INDEX idx_order_items_product
    ON order_items (product_id);
```

customer_segments_log — segmento atual e histórico

```
-- Segmento atual (query mais comum)
CREATE INDEX idx_segments_log_current
    ON customer_segments_log (customer_id, segment_id)
    WHERE is_current = true;

-- Análise de migração por período
CREATE INDEX idx_segments_log_date
    ON customer_segments_log (start_date DESC);
```

payments — constraint parcial para integridade

```
-- Garante no máximo 1 pagamento aprovado por pedido
-- sem bloquear retentativas com outros status
CREATE UNIQUE INDEX idx_payments_order_approved
    ON payments (order_id)
    WHERE status = 'approved';
```

inventory — alerta de reposição

```
-- Filtra apenas produtos abaixo do reorder_level
-- (query de estoque crítico executada com alta frequência)
CREATE INDEX idx_inventory_reorder
    ON inventory (product_id, warehouse_id)
```

```
    WHERE quantity_available < reorder_level;
```

7.3 Estratégia de Particionamento (orders)

Com 3M+ de linhas em 5 anos, a tabela orders se torna candidata ao particionamento por range de order_date. O PostgreSQL suporta partition pruning nativo — queries com filtro de período consultam apenas a partição relevante, reduzindo o custo de I/O em ordens de magnitude.

```
-- Particionamento por ano (adequado para e-commerce com sazonalidade)
CREATE TABLE orders (
    order_id          bigint         NOT NULL,
    order_date        timestamp      NOT NULL,
    -- demais colunas...
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024
    PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

CREATE TABLE orders_2025
    PARTITION OF orders
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

> **Quando aplicar:** Particionamento compensa a partir de ~5M linhas com queries frequentes por período. Abaixo disso, índice em order_date é suficiente e mais simples de manter.

7.4 Considerações para Camada Analítica

Para relatórios de alta frequência (dashboards, exports diários), recomenda-se mover a carga analítica para fora do banco transacional por meio de uma das seguintes estratégias:

Estratégia	Quando usar
Materialized Views	Relatórios que toleram dados com até 1h de defasagem. Simples de implementar no próprio PostgreSQL com REFRESH MATERIALIZED VIEW CONCURRENTLY.
Read Replica	Dashboards em tempo quase real que não podem impactar a escrita transacional. Replica assíncrona do PostgreSQL.
Data Warehouse (ex.: BigQuery, Redshift)	Quando o volume ultrapassa 50M linhas ou há necessidade de joins com fontes externas (CRM, ERP, marketing). Exige pipeline ETL/ELT.

Estratégia	Quando usar
dbt (data build tool)	Para modelar camadas analíticas (staging → marts) sobre o PostgreSQL ou DW, versionando as transformações como código.